Tuesday 14/4/26

LECTURE-17

- Counting Semaphores.

- Dead Locks : P1 holding ↑ A (resource) and needs B ; P2 holding B ↑ (resource) and wants A. Circular deadlock.

* Semaphore mistakenly can cause deadlocks like prev. example lecture.

⇒ **Resource type**: Have all instance which are similar (so we can use any)

- If process requires any instance from resource type and it does not gets that instance, that means resource type isn't identical, hence it isn't ↑ (resource type) (e.g: 3 Simple, 1 color printer) → we can ask simple printer & get color.

⇒ **Characteristics of DeadLocks**: (All conditions must fulfil)

1. **Mutual Exclusion**: There is at least one resource present which is non shareable (need semaphore for critical section) (memory, printer, semaphore resource)
2. **Hold and wait**: If process has one or more resources & it demands more ↑
3. **No. Preemption**: Process will voluntarily give up its right to use processor and it can not be snached
4. **Circular wait**: (above first line example).

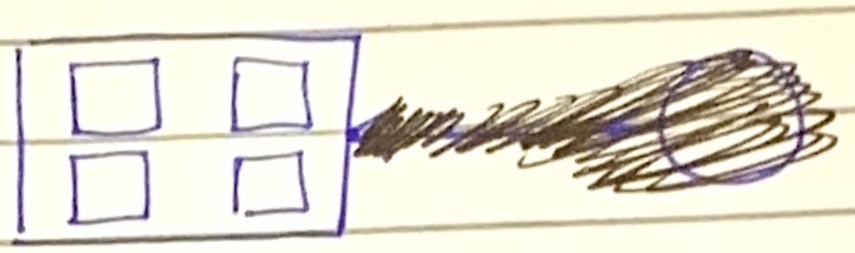
4. Circular

1. if R1 is Shareable then P2 does not needs to wait for R2 ↑
2. If no hold & wait, circular wait won't happen
3. If pre-emption allowed then CPU can take/pre-empt the resource (R1 & R2) stuck in loop and taking it and giving to other process would end loop.

⇒ Resource allocation Graph:

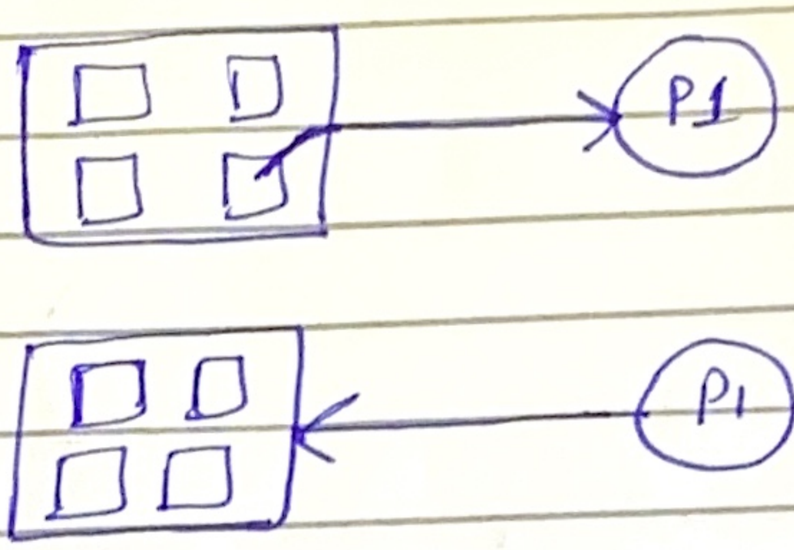
Process : ○

Resource type with 4 instances



Process P1 request resource type

Resource type allocates an instance to P1

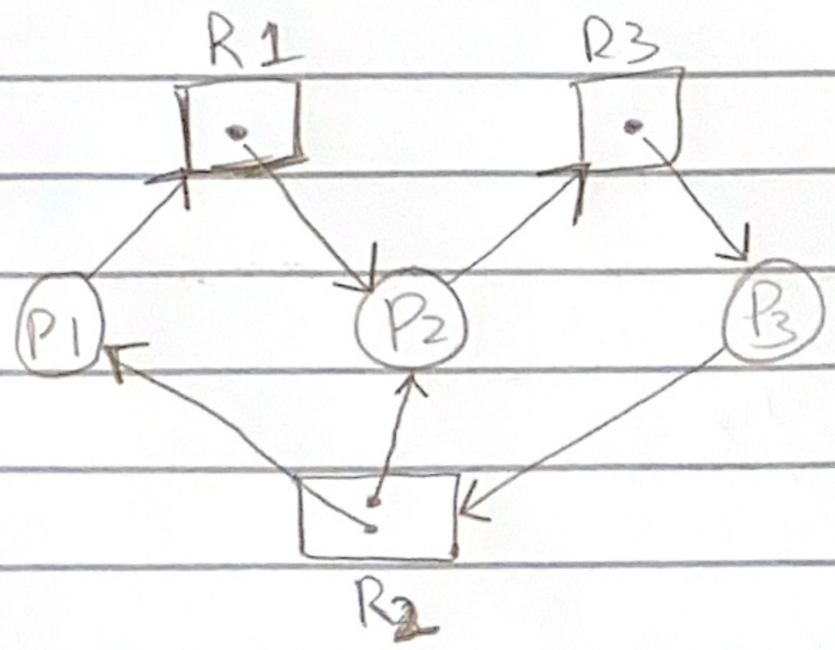


⇒ CHECK Deadlocks from graph.

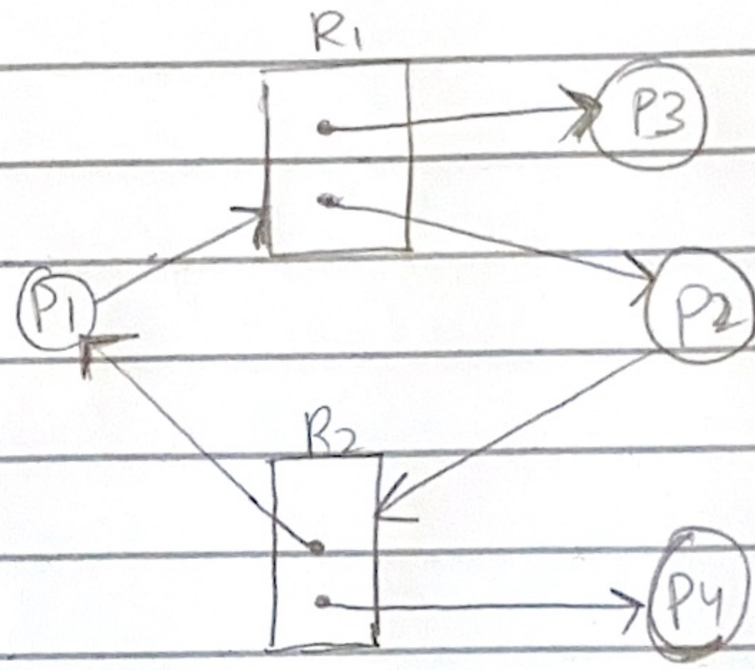
IF Cycle doesnot exist → No deadlock

IF Cycle exist → IF single instance in resource → Deadlock

→ IF multiple instance in resource → Could be deadlock.



- Cyclic : R1 → P2 → R3 → P3 → R2 → P1 → R1
- Only R2 is multiple instance & it shows deadlock as upper instance of R2 is held by P2 and P2 wants R3 whereas R3 is held by P3 which wants R2 i.e: not available (same with lower instance)



- Cyclic: R1 → P2 → R2 → P1 → R1.
- NO deadlock. R1 is multiple instance and so is R2. In R1; the upper instance is held by P3 but it'll be released once P3 is done thus "HOLD & WAIT" condition for deadlock fails
- Same for lower instance of R2 & P4.